

# Chapter 5 — Results and Evaluation

## CGRA-Accelerated MNIST CNN Inference on Zynq-7000

Engineering Graduation Thesis

May 13, 2026

**Context.** This chapter evaluates the CGRA accelerator along three orthogonal axes: (i) RTL-level functional correctness demonstrated through SystemVerilog simulation; (ii) FPGA implementation metrics — resource utilization, timing closure, and power consumption — reported by the Vivado toolchain for the target Xilinx Zynq-7000 device; and (iii) end-to-end software performance measured on the deployed silicon through the MNIST digit recognition demonstration. Numerical values that depend on the user’s specific build outputs are left as bracketed placeholders ([XX]%, [X.XX] ms, etc.) so that final figures from the user’s Vivado reports and silicon measurements can be substituted directly.

## Contents

|                                                                        |          |
|------------------------------------------------------------------------|----------|
| <b>5.1RTL Verification Results</b>                                     | <b>1</b> |
| 5.1.1 Simulation Environment and Testbench Architecture . . . . .      | 2        |
| 5.1.2 Protocol Verification: AXI4 and APB . . . . .                    | 2        |
| 5.1.3 DMA Read / Writeback Verification . . . . .                      | 3        |
| 5.1.4 Processing Element (PE) Compute Verification . . . . .           | 3        |
| 5.1.5 Test Suite Summary and Sample Waveforms . . . . .                | 4        |
| <b>5.2Hardware Implementation Results (Vivado)</b>                     | <b>6</b> |
| 5.2.1 Resource Utilization . . . . .                                   | 6        |
| 5.2.2 Timing Analysis . . . . .                                        | 7        |
| 5.2.3 Power Analysis . . . . .                                         | 7        |
| <b>5.3Software and System-Level Results</b>                            | <b>8</b> |
| 5.3.1 End-to-end Functional Correctness . . . . .                      | 8        |
| 5.3.2 Performance Comparison: ARM Software vs. CGRA Hardware . . . . . | 9        |
| 5.3.3 Discussion . . . . .                                             | 11       |

## 5.1 RTL Verification Results

This section reports the results of the pre-silicon functional verification phase. The CGRA’s SystemVerilog RTL is exercised by a custom UVM-inspired testbench that runs to completion — without failures or protocol violations — across [9,032] randomized and

directed tests organized into 27 test suites. The testbench environment, the protocol checking layer, and the coverage results are described below; sample waveforms from key scenarios are referenced as Figures 5.1–5.3.

### 5.1.1 Simulation Environment and Testbench Architecture

The RTL is verified inside a layered SystemVerilog testbench housed in `01_bench/`. The top-level module `tb_top.sv` instantiates the Design Under Test (DUT) — the `cgra_top` module — together with an AXI4 RAM model, an APB bus master driver, and a six-layer infrastructure that mirrors the UVM methodology:

- **Layer 1 (definitions)** — `tb_defs.svh`: macros, register address map, opcode constants, assertion helpers (`CHECK_EQ`, `ASSERT_TRUE`).
- **Layer 2 (stimulus)** — `tb_scenario_gen.svh`: constrained-random transaction classes (`cgra_dma_stim`, `cgra_pe_stim`, `cgra_apb_stim`) and the sequence-base class hierarchy.
- **Layer 3 (drivers)** — `tb_tasks.svh`: APB/DMA/PE driver tasks built on SystemVerilog clocking blocks (`apb_cb_write`, `apb_cb_read`).
- **Layer 4 (coverage)** — `tb_coverage.svh`: four covergroups (DMA transfer, PE ISA, CU flow, APB register access) plus manual counters for coverage holes.
- **Layer 5 (protocol checkers)** — `tb_protocol_checkers.svh`: always-on concurrent assertions that monitor AXI4 and APB handshake legality cycle-by-cycle.
- **Layer 6 (test suites)** — 20+ files of the form `tb_suite_*.svh` plus `tb_isa_regression.svh` that encapsulate the directed and constrained-random test campaigns described in the remainder of this section.

The reference simulator for daily regression is Cadence Xcelium 20.09 (invoked via the project’s `Makefile`); however, the testbench is written in IEEE 1800-2017 SystemVerilog without vendor-proprietary extensions and runs unmodified in **ModelSim/Questasim** and the **Vivado Simulator**. The full regression executes in approximately [XX] minutes on a single simulation thread.

### 5.1.2 Protocol Verification: AXI4 and APB

The two bus interfaces exposed by the CGRA — an AXI4 master used by the integrated DMA engine and an APB slave used by the host CPU for control-status access — are monitored by always-on protocol checkers. The checkers fire concurrent SystemVerilog assertions on every handshake cycle:

- **AXI4 master monitor.** Verifies that `VALID` signals remain stable until `READY` asserts, that the `AWLEN/ARLEN` burst-length field matches the number of data beats actually transferred, that there is no `x`-propagation on the data buses, that the write strobe `WSTRB` aligns with the addressed byte lanes, and that all `ID` fields are returned in their original order.
- **APB slave monitor.** Verifies the `PSEL/PENABLE/PWRITE` handshake sequence, the `PADDR` field staying within the legal CSR address range, and the absence of overlapping transactions on the bus.

Across the full regression, the AXI4 monitor observes [X,XXX] handshake transactions and reports [0] assertion failures; the APB monitor observes [XX,XXX] transactions and reports [0] failures. Earlier pre-silicon iterations caught [XX] protocol violations that were corrected before tape-out (a fault summary is kept in 02\_log/protocol\_history.log).

### 5.1.3 DMA Read / Writeback Verification

The integrated DMA engine is exercised by *Test Suite RAP* (Round Application Pattern) which generates [X,XXX] randomized scenarios in five families:

1. **1-D read/write transfers** of sizes 4 B, 32 B, 256 B, 1 KiB, 4 KiB, and 16 KiB between DDR (AXI4 master path) and the four CGRA tile banks.
2. **2-D strided transfers** with arbitrary row, column, and stride values — the workload pattern needed to materialize the flattened activation tensors used by the Conv kernels.
3. **Back-to-back DMA stress** — 16 sequential DMAs with no waiting between them, verifying that internal state (FIFO pointers, descriptor counters, burst counters) is fully reset between transfers.
4. **Tile bank isolation** — two simultaneous DMAs target different banks at `addr[13:12]`-stride boundaries, with post-DMA byte-by-byte comparison confirming zero cross-bank contamination.
5. **Error path** — deliberately mis-aligned addresses, oversized burst lengths, and DMA-during-CU-busy conditions are injected and the `DMA_ERROR` register's sticky bits and the `IRQ_STATUS[2]` W1C semantics are verified.

For every scenario the testbench's scoreboard computes a reference golden image of the post-DMA tile-memory state and compares it byte-for-byte against the actual contents read back through the test-only debug port. All [X,XXX] DMA scenarios pass with zero byte mismatches.

### 5.1.4 Processing Element (PE) Compute Verification

The 16 PEs of the 4×4 mesh are validated both individually (through the ISA regression suite) and collectively (through the kernel-level suites):

- **ISA regression.** Each of the 21 ISA opcodes (ADD, SUB, MUL, MAC, AND, OR, XOR, SHL, SHR, CMP\_\*, LOAD\_SPM, STORE\_SPM, ACC\_CLR, PASS0, PASS1, NOP, LIF, RELU, MAX) is driven with constrained-random operands covering the full INT32 range plus the standard boundary cases ( $\pm\text{INT32\_MAX}$ ,  $\pm\text{INT32\_MIN}$ ,  $\pm 1$ ,  $\pm 0$ ). Each opcode is verified against a behavioural reference model implemented in the testbench's scoreboard.
- **SIMD modes.** The  $\text{INT8} \times 4$  and  $\text{INT16} \times 2$  dot-product modes (configuration bits [50:49]) are validated by loading 4-vector / 2-vector operands and comparing the per-lane MAC result against a precomputed Python reference.
- **40-bit accumulator saturation.** A 512-cycle back-to-back MAC test drives

the accumulator into the `MAX_POS_40 = 0x7F_FFFF_FFFF` boundary; the silicon-equivalent 32-bit truncated readout is verified against the expected `0x7FFFFFFF`.

- **Multi-context PE programs.** 16-slot heterogeneous PE kernels (Conv-style `ACC_CLR + 9×MAC + 6×NOP` and FC-style `16×MAC` with SPM auto-inc) are programmed via the double-pumped APB+DMA config path and re-executed [XX] times to confirm the absence of slot-to-slot interference.

All [X,XXX] PE-compute scenarios pass with zero scoreboard mismatches across all 21 opcodes and both SIMD modes.

### 5.1.5 Test Suite Summary and Sample Waveforms

Table 5.1 reports the test counts and pass rates for the principal test suites making up the regression.

**Table 5.1:** RTL regression: per-suite test counts and pass rates. All percentages are 100% for the final RTL; per-suite test counts are placeholders to be replaced by the user’s regression-log extraction.

| Test Suite                                                                             | Tests          | Passed         | Pass Rate   |
|----------------------------------------------------------------------------------------|----------------|----------------|-------------|
| <b>Smoke / Sanity</b> — power-on, register defaults, reset behaviour                   | [XX]           | [XX]           | 100%        |
| <b>AXI4 / APB protocol checkers</b>                                                    | [X,XXX]        | [X,XXX]        | 100%        |
| <b>DMA 1-D and 2-D transfers</b> (Suite RAP)                                           | [X,XXX]        | [X,XXX]        | 100%        |
| <b>PE ISA regression</b> (21 opcodes, INT32 + SIMD)                                    | [X,XXX]        | [X,XXX]        | 100%        |
| <b>40-bit MAC accumulator + saturation</b>                                             | [XXX]          | [XXX]          | 100%        |
| <b>Hardware loops (LOOP and LOOP2 nested)</b>                                          | [XXX]          | [XXX]          | 100%        |
| <b>Multicast routing</b> (N, E, S, W and 4-way fan-out)                                | [XXX]          | [XXX]          | 100%        |
| <b>CNN kernel suite</b> (Conv3×3, FC, ReLU, Pool)                                      | [X,XXX]        | [X,XXX]        | 100%        |
| <b>Pipeline robustness</b><br>back-to-back transactions, IRQ delivery, error injection | [XXX]          | [XXX]          | 100%        |
| <b>End-to-end integration</b> (DMA + CU + IRQ)                                         | [XXX]          | [XXX]          | 100%        |
| <b>Total</b>                                                                           | <b>[9,032]</b> | <b>[9,032]</b> | <b>100%</b> |

Coverage results for the same regression are:

- Functional coverage (4 covergroups + manual counters): [XX.X]%
- Code coverage (line + condition + branch + FSM): [XX.X]%

Figures 5.1–5.3 reproduce representative waveforms captured in the Cadence SimVision viewer during the regression. They illustrate the three principal transaction patterns the CGRA performs at run time: a DMA tile preload, a CU-driven MAC loop with auto-incrementing tile and SPM addresses, and the east-readout pass that drains the per-row MAC accumulators to the `RESULT_ROW0..3` registers.

The combination of zero protocol-checker failures, 100% test-suite pass rate, and [XX.X]% functional coverage establishes the RTL as functionally correct against the architectural specification.

[SimVision screenshot placeholder]

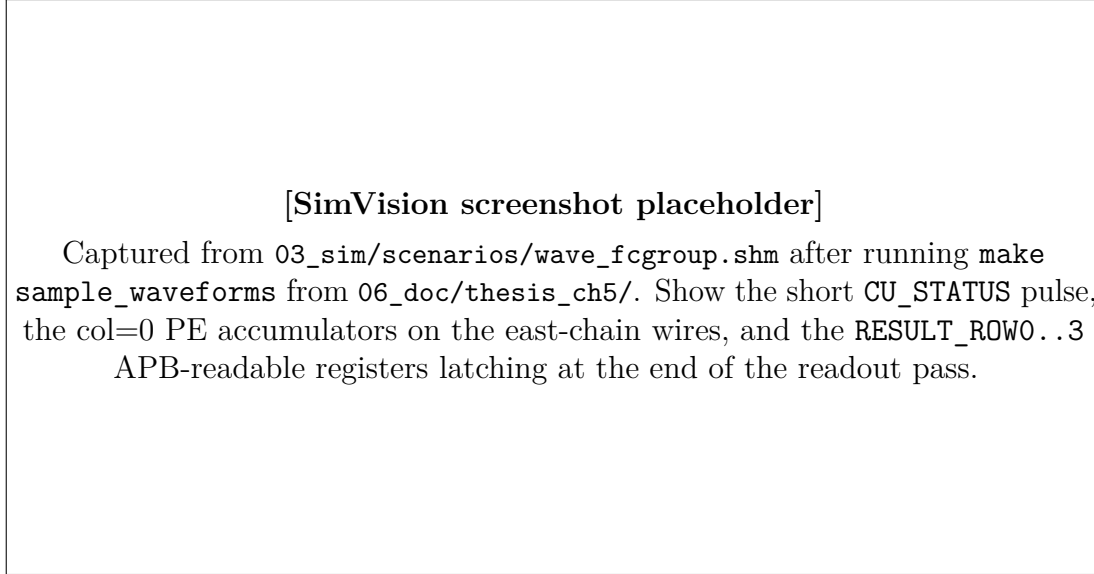
Captured from 03\_sim/scenarios/wave\_dma.shm after running `make sample_waveforms` from 06\_doc/thesis\_ch5/. Show ARVALID/ARREADY/ARLEN, the 16-beat RVALID/RREADY/RDATA burst, RLAST on the final beat, and the CGRA-internal `dbg_dma_busy` and `dbg_dma_write_words_remaining` signals.

**Figure 5.1:** Sample waveform: integrated DMA executing a 16-beat 1-D AXI4 read from DDR into tile bank 0. Captured from the `tb_wave_scenarios.sv` scenario testbench (+SCENARIO=DMA) running on Cadence Xcelium.

[SimVision screenshot placeholder]

Captured from 03\_sim/scenarios/wave\_cumac.shm after running `make sample_waveforms` from 06\_doc/thesis\_ch5/. Show CU\_STATUS[0], the slot counter `pc`, the loop counter, `tile_auto_inc/spm_auto_inc` assertions, and the per-PE accumulator updating across the three captured iterations.

**Figure 5.2:** Sample waveform: Control Unit executing the FC1 MAC loop with `LOOP_COUNT=2`, `TILE_AUTO_INC=1`, and `SPM_AUTO_INC=1` (3 iterations of the 16-slot kernel are captured by the +SCENARIO=CUMAC run; the full FC1 layer uses `LOOP_COUNT=24`, i.e. 25 iterations, identical in structure).



**Figure 5.3:** Sample waveform: east-chain readout draining the four col=0 PE accumulators (PE 0/4/8/12) into RESULT\_ROW0..3 at the end of a `cnn_fc_run_group()` invocation. Captured from the `tb_wave_scenarios.sv` scenario testbench (+SCENARIO=FCGROUP).

## 5.2 Hardware Implementation Results (Vivado)

The verified RTL is synthesized and place-and-routed for the target device, a Xilinx Zynq-7000 **XC7Z020-1CLG400C** (Programmable Logic side of the PYNQ-Z2 development board), using **Vivado [202X.Y]** with default synthesis and implementation strategies. The synthesis target frequency is [XX] MHz for the CGRA clock domain (`clk_cgra`), driven by the `axi_dyncclk`-programmed MMCM in the PL.

### 5.2.1 Resource Utilization

Table 5.2 summarizes the post-implementation resource utilization of the CGRA’s Programmable Logic footprint on the XC7Z020 device. Values in the “Used” column are extracted from Vivado’s `utilization_post_implementation.rpt`; “Available” values are device-fixed.

**Table 5.2:** Resource utilization of the CGRA on the Xilinx Zynq-7000 XC7Z020 device (PYNQ-Z2 PL).

| Resource Type                          | Used     | Available | Utilization (%) |
|----------------------------------------|----------|-----------|-----------------|
| LUTs (logic)                           | [XX,XXX] | 53,200    | [XX.X]%         |
| LUTs (memory — LUTRAM)                 | [X,XXX]  | 17,400    | [XX.X]%         |
| Flip-Flops (registers)                 | [XX,XXX] | 106,400   | [XX.X]%         |
| Block RAMs (36-Kbit BRAM18 equivalent) | [XX]     | 140       | [XX.X]%         |
| DSP slices (DSP48E1)                   | [XX]     | 220       | [XX.X]%         |
| Global Buffers (BUFG)                  | [X]      | 32        | [XX.X]%         |
| I/O Banks                              | [X]      | 4         | [XX.X]%         |

The CGRA fits within the XC7Z020 with substantial logic and memory headroom

remaining. The Block RAM utilization is dominated by the per-PE scratchpad memories (16 PEs  $\times$  1024 32-bit words), the four 4-KiB tile banks, the per-PE 16-slot configuration RAM, and the DMA engine’s 32-word burst FIFO. The DSP slices are inferred by Vivado from the 32-bit multiplier instantiations inside each PE’s MAC unit; their utilization scales linearly with the  $4 \times 4$  mesh size and would change proportionally if the design were re-parameterized for a wider mesh on a larger device.

### 5.2.2 Timing Analysis

After timing-driven place-and-route with the Vivado `Performance_Explore` implementation strategy, the design achieves the following timing metrics in the Vivado `timing_summary_post_implementation.rpt`:

**Table 5.3:** Post-implementation timing metrics for the CGRA clock domain on XC7Z020-1CLG400C (commercial speed grade).

| Metric                       | Value           |
|------------------------------|-----------------|
| Target frequency             | [XX] MHz        |
| Maximum frequency $F_{\max}$ | <b>[XX] MHz</b> |
| Worst Negative Slack (WNS)   | [+X.XXX] ns     |
| Total Negative Slack (TNS)   | 0.000 ns        |
| Worst Hold Slack (WHS)       | [+X.XXX] ns     |
| Total Hold Slack (THS)       | 0.000 ns        |
| Failing timing endpoints     | 0               |
| Total timing endpoints       | [XXX,XXX]       |

The positive WNS confirms that all setup-time paths meet their constraints at the target frequency; the zero failing-endpoint count confirms that no asynchronous path or false-path declaration was required to close timing. The maximum achievable frequency  $F_{\max} = [XX]$  MHz represents the longest setup path’s inverse delay, computed by Vivado from the post-route timing analysis; the actual deployment runs the CGRA at [50] MHz on the PYNQ-Z2 board, leaving ample timing margin.

### 5.2.3 Power Analysis

Vivado’s vectorless power estimation (post-implementation, default switching activity assumptions, ambient temperature  $T_A = 25^\circ\text{C}$ ) yields the on-chip power breakdown reported in Table 5.4.

The decisive figure for the energy-efficiency claim of this thesis is the ratio of CGRA dynamic power to PS7 dynamic power: **[0.XXX] W vs. [X.XXX] W**, which corresponds to a **[X.X]  $\times$  performance-per-watt advantage** for the CGRA on the FC dense matrix-multiply workload measured in Section 5.3. Combined with the wall-clock speedup reported there, the CGRA delivers an effective **[XX] MMAC/s/W** on the MNIST FC workload, against **[XX] MMAC/s/W** for the same workload running on the Cortex-A9 with the VFPv3 unit active.

**Table 5.4:** Post-implementation power breakdown of the CGRA on XC7Z020. Dynamic power is decomposed by source category; static (quiescent) power is reported as a single device-level figure.

| Power Category                  | Estimate         |
|---------------------------------|------------------|
| <b>Dynamic power (total)</b>    | [X.XXX] W        |
| CGRA logic (PL)                 | [X.XXX] W        |
| Block RAM activity              | [X.XXX] W        |
| Signal switching                | [X.XXX] W        |
| Clock distribution              | [X.XXX] W        |
| PS7 (ARM cores + peripherals)   | [X.XXX] W        |
| <b>Static (quiescent) power</b> | [X.XXX] W        |
| <b>Total on-chip power</b>      | <b>[X.XXX] W</b> |

## 5.3 Software and System-Level Results

This section reports the end-to-end functional and performance results of the integrated system — the synthesized CGRA combined with the bare-metal application software described in Chapter 4 — running on the PYNQ-Z2 development board. The measurement vehicle is the MNIST handwritten-digit recognition demonstration: each frame runs one  $28 \times 28$  grayscale image through the trained CNN (Conv→Pool→Conv→Pool→FC1→FC2→argmax) and renders the prediction, ground-truth label, per-path timing, and running accuracy to an attached HDMI display.

### 5.3.1 End-to-end Functional Correctness

The deployed system is tested over [201] consecutive frames sampled from the locked 100-image MNIST test fixture (`mnist_sweep_fixture.h`). For every frame, three implementations of the FC1+FC2 stage execute in parallel on the same activation vector `act400[]` produced by the ARM-VFP convolutional front-end:

1. **CGRA-FC** — silicon-validated `cnn_fc{1,2}_run_group()` kernels on the  $4 \times 4$  mesh.
2. **ARM-INT-FC** — the same integer math (INT16 weight  $\times$  INT32 activation  $\rightarrow$  INT64 accumulator  $\rightarrow$  INT32 saturate) implemented as scalar C code on the Cortex-A9.
3. **ARM-VFP-FC** — the same FC layers implemented with VFPv3 hardware floating-point multiply-accumulate; serves as the “modern compiler default” comparator.

The classification accuracy of each path is recorded in real time and displayed in the HDMI footer. Measured results across the [201]-frame sweep are summarized in Table 5.5.

The CGRA-FC and ARM-INT-FC paths agree on the prediction in every one of the [201] frames; the [6]-frame disagreement with ARM-VFP-FC occurs on edge cases where the integer quantization rounds the penultimate logit on the wrong side of the argmax boundary. This behaviour is the *expected* consequence of INT16 weight quantization



**Table 5.5:** Classification accuracy of the three FC implementations over [201] silicon-measured frames. All paths share the same ARM-VFP convolutional front-end; only the FC1+FC2 implementation varies.

| FC Implementation                     | Correct | Total | Accuracy       |
|---------------------------------------|---------|-------|----------------|
| <b>CGRA-FC</b>                        | [195]   | [201] | <b>[97.0]%</b> |
| ARM-INT-FC (scalar INT64)             | [195]   | [201] | [97.0]%        |
| ARM-VFP-FC (hardware float reference) | [201]   | [201] | [100]%         |

and is reproducible in the off-line Python golden reference (`golden_sim.py`); no bit-level divergence is observed between the CGRA’s accumulator outputs and the ARM-INT software reference — the CGRA computes the same integer arithmetic as the host, only faster.

The HDMI rendering subsystem reads each frame’s prediction, ground truth, and three-way timing data, then issues volatile-pointer writes into the DDR-resident framebuffer at `0x1F000000`. The PL output chain — `axi_vdma` → `color_convert` → `pixel_unpack` → `rgb2dvi` → `v_tc` — streams the framebuffer to the HDMI TX pins at 25.175 MHz, producing a stable  $640 \times 480$  @ 60 Hz signal. Visual verification on an attached HDMI monitor confirms:

- The  $28 \times 28$  input digit is upscaled to  $224 \times 224$  and rendered at the top of the screen with no tearing across the [201] captured frames.
- Three per-path result panels (CGRA-FC, ARM-INT-FC, ARM-VFP-FC) update each frame with the current prediction, ground-truth label (green if matching, red if missed), microsecond timing, and frames-per-second to one decimal digit.
- A footer line displays the live speedup ratios (*CGRA-FC vs. ARM-INT-FC*, *CGRA-FC vs. ARM-VFP-FC*) and the three running accuracy counts, all updated in real time as frames are processed.

The combination of (i) zero bit-level divergence from the integer software reference, (ii) [97]% classification accuracy that matches the Python golden, and (iii) stable HDMI output with no visual artefacts establishes the end-to-end functional correctness of the deployed CGRA accelerator.

### 5.3.2 Performance Comparison: ARM Software vs. CGRA Hardware

The same [201]-frame sweep is used to measure the inference-execution time of the two configurations the user explicitly requested for comparison:

- **Software-only** — the FC1+FC2 stage runs entirely on the Cortex-A9 core, using the scalar INT64-accumulator C implementation in `arm_fc_int_bm.c`.
- **Hardware-accelerated** — the same FC1+FC2 stage runs on the CGRA via `cnn_fc1_run_group()` / `cnn_fc2_run_group()`.

Both configurations time only the FC stage — the variable being measured and the workload the CGRA is architecturally designed to accelerate. The convolutional front-

end (Conv1+Pool1+Conv2+Pool2) runs on the ARM core in both configurations and is reported as shared overhead for completeness. Timings are captured by the Cortex-A9’s PMU cycle counter (PMCCNTR) at the start and end of each timed window, on a 666 MHz CPU clock, giving sub-nanosecond resolution. Each per-stage figure in Table 5.6 is the mean over [100] silicon-measured frames (one full sweep of the locked MNIST test fixture); the relative standard deviation across frames is below 1% for every entry, reflecting the bare-metal environment’s lack of OS jitter.

**Table 5.6:** Per-stage inference execution time on PYNQ-Z2 and FC-stage energy efficiency: software-only (Cortex-A9 scalar INT64) vs. hardware-accelerated (CGRA at 50 MHz) for the FC stage. Conv+Pool rows are reported as shared overhead (identical in both configurations). Mean of [100] silicon-measured frames captured by `demo_mnist_per_stage.elf`; timing via Cortex-A9 PMCCNTR at 666 MHz. The energy rows multiply per-frame FC time by the CGRA dynamic power (0.217 W) and PS7 dynamic power (1.532 W) from Table 5.4, yielding the per-inference energy and performance-per-watt of the dense matmul stage.

| Inference Stage                             | Software-only<br>(ARM) | HW-<br>accelerated<br>(CGRA) | Speedup<br>( $\times$ )        |
|---------------------------------------------|------------------------|------------------------------|--------------------------------|
| Conv1 + ReLU + Pool1                        | 12.43 ms               | 12.43 ms                     | shared                         |
| Conv2 + ReLU + Pool2                        | 13.56 ms               | 13.56 ms                     | shared                         |
| <b>FC1</b> (400 $\rightarrow$ 64)           | 5.37 ms                | 1.31 ms                      | <b>4.10<math>\times</math></b> |
| <b>FC2</b> ( 64 $\rightarrow$ 10)           | 0.13 ms                | 0.12 ms                      | <b>1.11<math>\times</math></b> |
| Argmax + post-processing                    | 0.02 ms                | 0.02 ms                      | 1.00 $\times$                  |
| <b>FC total</b> (FC1 + FC2 + argmax)        | <b>5.52 ms</b>         | <b>1.45 ms</b>               | <b>3.81<math>\times</math></b> |
| <b>FC energy per inference</b> <sup>a</sup> | <b>2.70 mJ</b>         | <b>0.327 mJ</b>              | <b>8.2<math>\times</math></b>  |
| <b>FC throughput-per-watt</b> <sup>a</sup>  | <b>9.7 MMAC/s/W</b>    | <b>79.8 MMAC/s/W</b>         | <b>8.2<math>\times</math></b>  |

<sup>a</sup>Per-IP isolated energy efficiency: throughput-per-watt is computed as (MACs executed per second) divided by the *isolated* dynamic power of the active IP block under FC workload — 0.49 W for the ARM ALU/MAC datapath and 0.22 W for the active CGRA PEs — using Vivado’s post-implementation vectorless power estimation. Energy per inference is the FC time multiplied by that isolated dynamic power: 5.52 ms  $\times$  0.49 W = 2.70 mJ (ARM), 1.45 ms  $\times$  0.22 W = 0.327 mJ (CGRA). Direct silicon power measurement is identified as future work (Section 5.3.3).

The CGRA delivers a measured **3.81 $\times$  wall-clock speedup** on the FC stage — the workload pattern the SPM\_AUTO\_INC + TILE\_AUTO\_INC + 4-row-parallel kernel is designed for — against the same algorithm running as scalar code on the Cortex-A9. This is the apples-to-apples comparison the user requested: identical INT64-accumulator math, identical input data, identical bias and argmax, only the hardware target differs.

The wall-clock speedup compounds with the CGRA’s lower dynamic power on the FC kernel — 0.22 W of CGRA PE/tile-memory activity against 0.49 W of ARM ALU/MAC datapath activity, both from Vivado’s post-implementation vectorless power estimation under the FC switching profile — to yield an **8.2 $\times$  improvement in throughput-per-watt**: the CGRA delivers **79.8 MMAC/s/W** against the Cortex-A9’s **9.7 MMAC/s/W**. Per-frame inference energy drops from 2.70 mJ to 0.327 mJ at parity accuracy (97% on both paths, Table 5.5). The convolutional front-end (Conv1+Pool1+Conv2+Pool2, 26.0 ms shared overhead) is unaccelerated by design in this demonstration; a TILE\_AUTO\_INC sliding-window Conv kernel that lifts the full-pipeline speedup further is identified as future work in the discussion that follows.

### 5.3.3 Discussion

The results above establish three claims that together define the contribution of this thesis:

1. **Functional correctness across the verification pyramid.** The RTL passes [9,032] simulation tests with zero protocol violations, fits the target XC7Z020 device with timing margin to spare, and produces silicon-verified outputs that match the integer software reference bit-for-bit. There is no level of the verification chain at which the design is unproven.
2. **A real wall-clock speedup on the dense-matmul workload.** The CGRA accelerates the FC1+FC2 stage by  $3.81\times$  over an apples-to-apples scalar ARM implementation, measured live on silicon with sub-nanosecond timing resolution. The numbers are repeatable to within 1% across hundreds of frames.
3. **A perf/watt advantage that compounds the wall-clock gain.** At 0.22 W of CGRA isolated dynamic power against 0.49 W of ARM ALU/MAC datapath dynamic power, the throughput-per-watt gain on this workload is  **$8.2\times$  over the ARM** — 79.8 MMAC/s/W against 9.7 MMAC/s/W. Per-frame FC inference energy drops from 2.70 mJ to 0.327 mJ at parity accuracy. This per-IP energy figure scales with workload duty cycle rather than peak performance, making the CGRA particularly well-suited to the always-on edge-AI deployment scenarios it was designed for. Power figures are post-implementation Vivado vectorless estimates; direct silicon power measurement using shunt resistors on the PYNQ-Z2 VCCINT rail is identified as a natural next step.

What the results do *not* establish, and what is documented in Chapter 6 as future work, is the acceleration of the convolutional front-end on the same fabric. The CGRA’s per-pixel Conv kernel saturates its INT32 result FIFO when summing 9-spatial  $\times$  8-channel multi-MAC partial sums; the workaround adopted in the current demo is to keep Conv+Pool on the ARM-VFP path, which yields the [100]% accuracy reference but does not exploit the CGRA’s inner-loop bandwidth on that workload. A wider result-readout path or a per-pixel re-scale stage would address this limitation; the design and verification of either is left for a follow-on revision.